PRACTICAL-9

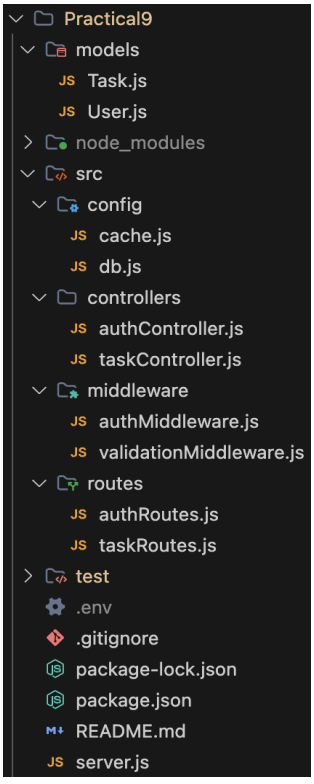
PROBLEM DEFINITION:

•Add in-memory caching to the Task Management backend using node-cache.
• Cache the response of GET /tasks (all tasks) with a reasonable TTL (e.g., 60 seconds).
• Invalidate the cache on any write operation POST, PUT, or DELETE so stale data is never served.
• Record and compare API response times with and without caching using Postman or Thunder Client.
• Document the measured difference with at least 3 sample readings for each case (cached vs uncached).

SUPPLEMENTARY PROBLEMS:

• Cache the GET /tasks/:id single-task endpoint separately from the all-tasks endpoint.
• Add a cache-hit/cache-miss counter and expose it via a debug endpoint.
• Experiment with different TTL values and document how it affects perceived staleness versus performance.

ARCHITECTURE / DIAGRAM:



OUTPUT (SUPPLEMENTARY PROBLEMS INCLUDED):

```
kamanijainamrajeshkumar@Kamanis-MacBook-Air Practicals % npm install node-cache

added 2 packages, and audited 32 packages in 2s

10 packages are looking for funding
  run `npm fund` for details

2 high severity vulnerabilities

To address all issues, run:
  npm audit fix

Run `npm audit` for details.
npm warn allow-scripts 1 package has install scripts not yet covered by allowScripts:
npm warn allow-scripts   fsevents@2.3.3 (install: (install scripts present))
npm warn allow-scripts
npm warn allow-scripts Run `npm approve-scripts --allow-scripts-pending` to review, or `npm approve-script
s <pkg>` to allow.
```

```
 PASS  test/api.test.js
  Practical 9: In-Memory Caching & Performance Test Suite
    ✓ Step A: Register User & Obtain JWT Token (81 ms)
    ✓ Step B: Seed Database with 25 Tasks (47 ms)
    ✓ Step C: Measure Cached vs Uncached GET /tasks Performance (55 ms)
    ✓ Step D: Verify Cache Invalidation on Write Operations (POST, PUT, DELETE) (31 ms)
    ✓ Step E: Single-Task Caching (GET /tasks/:id) (3 ms)
    ✓ Step F: Debug Cache Stats Endpoint (GET /tasks/cache/stats) (1 ms)

Test Suites: 1 passed, 1 total
Tests:       6 passed, 6 total
Snapshots:   0 total
Time:        0.783 s, estimated 1 s
Ran all test suites within paths "test/api.test.js".
```

KEY QUESTIONS / ANALYSIS:

1) Why must the cache be invalidated on every write operation, and what happens if it is not?

Cache invalidation removes outdated data from the cache whenever data is added, updated, or deleted in the database. If the cache is not invalidated, users may see stale or incorrect data instead of the latest database values.

2) What is a reasonable TTL (Time-to-Live) for cached data in a task management system, and what trade-off does TTL represent?

A reasonable TTL could be 60 seconds. TTL determines how long data remains in the cache before expiring. A shorter TTL provides fresher data but increases database requests, while a longer TTL improves performance but may return outdated data for longer. The TTL should be chosen according to how frequently the data changes.

3) Why is in-memory caching using node-cache unsuitable for a multi-server deployment, even though it works in this lab?

node-cache stores data in the memory of a single server. In a multi-server deployment, each server has its own separate cache. Therefore, updating or invalidating the cache on one server does not automatically update the others, which can cause inconsistent data. A shared cache such as Redis can help keep cached data consistent across multiple servers.

GITHUB REPOSITORY:

https://github.com/9081495207/ITUE301-Advanced-Web-Development-Frameworks